

Universidade do Minho

Escola de Engenharia

Mestrado em Inteligência Artificial

Otimização Multiobjetivo de Percursos Multimodais no Grande Porto

Projeto de Computação Inspirada na Natureza

Autores:

Diogo José Borges Dias PG60245 Diogo Lopes Azevedo PG61217

27 de dezembro de 2025

Resumo

A mobilidade urbana sustentável constitui um desafio central nas áreas metropolitanas modernas, exigindo soluções que equilibrem eficiência temporal e impacto ambiental. Este relatório apresenta o desenvolvimento de um sistema de planeamento de percursos multimodais para a Área Metropolitana do Porto, formulado como um problema de otimização multiobjetivo. O sistema considera simultaneamente a minimização do tempo de viagem e das emissões de dióxido de carbono (CO_2), integrando redes de Metro (Metro do Porto), Autocarro (Serviço de Transportes Coletivos do Porto) e deslocações pedonais. A resolução do problema baseia-se em algoritmos evolucionários multiobjetivo, com particular enfoque no algoritmo MOEA/D [10]. Nesta primeira parte do relatório são apresentados o enquadramento teórico, a formulação do problema e os fundamentos da otimização multiobjetivo necessários para a compreensão da abordagem proposta.

Conteúdo

1	Introdução	5
2	Enquadramento Teórico	6
2.1	Otimização em Problemas Reais	6
2.2	Otimização Multiobjetivo	6
2.3	Dominância de Pareto	6
3	Abordagens para Otimização Multiobjetivo	7
3.1	Abordagens por Pesos	7
3.2	Abordagens Baseadas em Pareto	7
4	Caracterização do Problema de Mobilidade	8
5	Formulação Matemática	9
5.1	Definição do Grafo e Variáveis	9
5.2	Objetivos	9
5.3	Otimização via MOEA/D	9
5.4	Restrições Adicionais de Viabilidade	10
6	Algoritmos Evolutivos para Planeamento de Percursos	11
6.1	Representação das Soluções	11
7	MOEA/D: Algoritmo Evolutivo Baseado em Decomposição	14
7.1	Decomposição do Problema e Vizinhança	14
7.2	Função de Agregação de Tchebycheff	14
7.3	Inicialização Heurística com Algoritmos de Caminho Mínimo	15
8	Operadores Genéticos Específicos para Grafos	17
8.1	Operador de Crossover por Nó Comum	17
8.2	Operador de Mutação com Recombinação de Segmentos	17
9	Atualização da População e Manutenção da Frente de Pareto	18
9.1	Mecanismo de Seleção Baseado em Vizinhanças (MOEA/D)	18
9.2	Manutenção da Frente de Pareto com Penalização	19
9.3	Poda por Epsilon-Dominance	19
10	Metodologia Experimental	21
10.1	Cenários de Teste	21
10.2	Configuração Experimental	21
11	Métricas de Avaliação	22
11.1	Convergência	22
11.2	Diversidade	22

12 Resultados Experimentais	23
12.1 Frente de Pareto Obtida	23
12.2 Soluções Representativas	24
13 Análise da Convergência	24
14 Discussão dos Resultados	25
14.1 Vantagens da Abordagem	25
14.2 Limitações	25
14.3 Ameaças à Validade	25
15 Conclusões e Trabalho Futuro	26
A Visualização da Rede Multimodal	27

Lista de Figuras

1	Excerto do grafo multimodal gerado no contexto do problema(disponível através do <code>export_graph.py</code>)	13
2	Compromisso entre tempo de viagem e emissões de CO ₂ para um caso de cada nível de dificuldade.	23
3	Distribuição dos tempos das soluções não dominadas para um caso de cada nível de dificuldade.	23
4	Evolução do tamanho da Frente de Pareto ao longo das gerações para um caso de cada nível de dificuldade.	24
5	Evolução do hipervolume ao longo das gerações para um caso de cada nível de dificuldade.	24
6	Representação geográfica das linhas do Metro do Porto utilizadas na construção do grafo multimodal.	27
7	Distribuição espacial das paragens de transporte público (Metro e Autocarro) no Grande Porto.	27

1 Introdução

O crescimento das áreas urbanas e metropolitanas tem conduzido a um aumento significativo da complexidade dos sistemas de mobilidade. Cidades como o Porto enfrentam diariamente o desafio de garantir deslocações eficientes para milhares de utilizadores, conciliando tempo de viagem, conforto, custo económico e impacto ambiental.

Tradicionalmente, os sistemas de planeamento de percursos focam-se na minimização de um único critério, o tempo de viagem. No entanto, esta abordagem ignora outros fatores relevantes, nomeadamente as emissões de gases de efeito de estufa, que assumem um papel cada vez mais central no contexto das políticas de sustentabilidade e mobilidade urbana [8, 9].

Neste contexto, o planeamento de percursos deve ser encarado como um problema de **otimização multiobjetivo**, no qual vários critérios, frequentemente contraditórios, devem ser considerados simultaneamente. Em particular, existe um compromisso evidente entre a rapidez de uma deslocação e o seu impacto ambiental.

O objetivo principal deste projeto é o desenvolvimento de um sistema de apoio à decisão que permita aos utilizadores do Grande Porto selecionar percursos multimodais, tendo em consideração:

- a minimização do tempo total de viagem;
- a minimização das emissões de dióxido de carbono (CO_2).

Para garantir a viabilidade prática e a aceitação das soluções por parte dos utilizadores, o modelo incorpora ainda restrições operacionais rigorosas. De forma a evitar percursos excessivamente fragmentados ou fisicamente exigentes, o sistema impõe um limite máximo de **4 transbordos** por viagem e restringe o tempo total de caminhada a um máximo de **90 minutos**.

A resolução deste problema recorre a técnicas de **Computação Inspirada na Natureza**, nomeadamente algoritmos evolucionários multiobjetivo, capazes de produzir um conjunto de soluções alternativas, representando diferentes compromissos entre os objetivos considerados.

2 Enquadramento Teórico

2.1 Otimização em Problemas Reais

Muitos problemas do mundo real caracterizam-se pela presença de múltiplos objetivos que não podem ser simultaneamente otimizados [3]. Exemplos comuns incluem:

- desempenho versus consumo energético;
- custo versus qualidade;
- tempo de execução versus precisão;
- tempo de viagem versus emissões de poluentes.

Nestes cenários, a existência de objetivos contraditórios impede a definição de uma única solução ótima global, tornando necessária a identificação de soluções de compromisso.

2.2 Otimização Multiobjetivo

Formalmente, um problema de otimização multiobjetivo pode ser definido como:

$$\min / \max \mathbf{f}(x) = (f_1(x), f_2(x), \dots, f_M(x))$$

sujeito a restrições:

$$\begin{aligned} g_j(x) &\geq 0, & j &= 1, \dots, J \\ h_k(x) &= 0, & k &= 1, \dots, K \end{aligned}$$

onde x representa uma solução no espaço de decisão.

Ao contrário da otimização mono-objetivo, a comparação direta entre soluções deixa de ser trivial, sendo necessário recorrer ao conceito de **dominância de Pareto** [2].

2.3 Dominância de Pareto

Assumindo um problema de minimização, uma solução x domina uma solução y (se escreve $x \prec y$) se, e só se:

- $f_i(x) \leq f_i(y)$ para todos os objetivos i ;
- existe pelo menos um objetivo j tal que $f_j(x) < f_j(y)$.

Uma solução é dita **não dominada** se não existir qualquer outra solução que a domine. O conjunto de todas as soluções não dominadas constitui o **Conjunto de Pareto**, enquanto a sua projeção no espaço dos objetivos é designada por **Frente de Pareto** [7].

A abordagem de Pareto apresenta várias vantagens:

- evita a agregação artificial de objetivos através de pesos fixos;
- fornece múltiplas soluções de elevada qualidade;
- permite ao decisor humano escolher o compromisso mais adequado.

3 Abordagens para Otimização Multiobjetivo

Existem várias estratégias para abordar problemas multiobjetivo.

3.1 Abordagens por Pesos

As abordagens baseadas em pesos combinam todos os objetivos numa única função:

$$f(x) = \sum_{i=1}^M w_i f_i(x)$$

onde w_i são pesos atribuídos a cada objetivo.

Apesar da sua simplicidade, estas abordagens apresentam limitações significativas:

- exigem a definição prévia de pesos adequados;
- não permitem obter múltiplos compromissos numa única execução;
- podem falhar na exploração de frentes de Pareto não convexas.

3.2 Abordagens Baseadas em Pareto

As abordagens baseadas em Pareto mantêm uma população de soluções e utilizam relações de dominância para guiar o processo evolutivo. Algoritmos como NSGA-II [4], SPEA2 e MOEA/D [10] inserem-se nesta categoria.

Estas abordagens são particularmente adequadas para problemas complexos, uma vez que:

- produzem um conjunto diversificado de soluções;
- mantêm explicitamente a noção de compromisso entre objetivos;
- são robustas a funções objetivo não lineares e não convexas.

4 Caracterização do Problema de Mobilidade

O problema abordado neste projeto consiste no planeamento de percursos multimodais numa rede urbana real, integrando:

- rede de Metro do Porto [1];
- rede de Autocarros da STCP [9];
- deslocações pedonais.

Cada percurso possível representa uma solução candidata, sendo avaliado de acordo com dois objetivos contraditórios:

1. tempo total de viagem;
2. emissão total de CO_2 .

Para garantir o realismo e a utilidade das soluções propostas, o modelo impõe ainda um conjunto de **restrições de viabilidade**. Ao contrário de um problema teórico de caminho mais curto, um percurso multimodal deve respeitar limites físicos e de conforto do utilizador:

- **Tempo Máximo de Caminhada:** Impede a geração de percursos que exijam esforço físico excessivo (ex.: caminhar vários quilómetros entre paragens);
- **Número Máximo de Transbordos:** Limita a complexidade da viagem, penalizando soluções que obriguem a trocas excessivas de modo de locomoção, que diminuem o conforto e aumentam a incerteza da viagem.

Este problema apresenta várias características que o tornam adequado à aplicação de algoritmos evolucionários:

- espaço de procura vasto e altamente combinatório;
- ausência de gradiente;
- múltiplos ótimos locais;
- necessidade de gerar múltiplas soluções alternativas.

5 Formulação Matemática

O problema de planeamento de percursos multimodais é modelado como um problema de caminho mais curto bi-objetivo num grafo orientado.

5.1 Definição do Grafo e Variáveis

Seja $G = (V, E)$ um grafo orientado, onde V é o conjunto de nós (paragens de autocarros e estações de metro) e E o conjunto de arestas. A cada aresta $e \in E$ estão associados os seguintes atributos:

- $t_e > 0$: tempo de viagem;
- $d_e > 0$: distância física;
- $c_e \geq 0$: emissão de CO_2 ;
- $m_e \in \{\text{metro, autocarro, pedonal}\}$: modo de transporte.

Seja $x = \{x_e \mid e \in E\}$ o vetor de variáveis de decisão, onde:

$$x_e = \begin{cases} 1 & \text{se a aresta } e \text{ faz parte do percurso,} \\ 0 & \text{caso contrário.} \end{cases} \quad (1)$$

5.2 Objetivos

O problema visa minimizar simultaneamente duas funções objetivo conflituosas:

- **Minimização do Tempo Total:**

$$T(x) = \sum_{e \in E} t_e \cdot x_e \quad (2)$$

- **Minimização das Emissões de CO_2 :**

$$C(x) = \sum_{e \in E} c_e \cdot x_e \quad (3)$$

O problema de otimização multiobjetivo formula-se então como:

$$\min_{x \in \mathcal{X}} (T(x), C(x)) \quad (4)$$

5.3 Otimização via MOEA/D

Para resolver este problema, utiliza-se o algoritmo MOEA/D, que decompõe o problema original em N subproblemas escalares. Cada subproblema é definido por um vetor de pesos $\lambda = (\lambda_1, \lambda_2)$, com $\lambda_i \geq 0$ e $\sum \lambda_i = 1$.

Utiliza-se a função de agregação de **Tchebycheff** para avaliar a qualidade de uma solução x num subproblema definido por λ :

$$g^{te}(x \mid \lambda, z^*) = \max_{j \in \{1,2\}} \{ \lambda_j \cdot |f_j(x) - z_j^*| \} \quad (5)$$

Onde $z^* = (z_1^*, z_2^*)$ é o ponto de referência ideal. Na prática, como o ótimo global é desconhecido, este ponto é **estimado dinamicamente** a partir da população atual (P) em cada geração:

$$z_j^* \approx \min_{x \in P} f_j(x), \quad j \in \{1,2\} \quad (6)$$

onde se assume $f_1(x) = T(x)$ e $f_2(x) = C(x)$.

5.4 Restrições Adicionais de Viabilidade

Para assegurar o realismo das soluções, podem ser aplicadas restrições adicionais ao espaço $\mathcal{X}$:

- **Limite de Caminhada:** A distância total percorrida a pé não deve exceder um limite $D_{\text{walk}}^{\text{max}}$:

$$\sum_{e \in E_{\text{pedonal}}} d_e \cdot x_e \leq D_{\text{walk}}^{\text{max}} \quad (7)$$

- **Limite de Transbordos:** O número de trocas de veículo (representado por arestas de transferência) é limitado a K_{transf} :

$$\sum_{e \in E_{\text{transf}}} x_e \leq K_{\text{transf}} \quad (8)$$

6 Algoritmos Evolutivos para Planeamento de Percursos

O problema em estudo apresenta um espaço de procura vasto, discreto e altamente combinatório, uma vez que cada solução corresponde a um caminho válido num grafo urbano multimodal com mais de 2500 paragens. Este tipo de problema é particularmente desafiante para métodos clássicos de otimização determinística, sobretudo devido à:

- natureza discreta do espaço de decisão (presença/ausência de arestas);
- ausência de informação de gradiente utilizável;
- existência de múltiplos ótimos locais;
- trade-off não-linear entre tempo de viagem e emissões de CO₂.

Os algoritmos evolucionários [5] surgem como uma abordagem adequada, uma vez que:

- operam sobre populações de soluções, permitindo a exploração simultânea de diferentes regiões do espaço de procura;
- não requerem informação de gradiente ou propriedades de continuidade;
- equilibram exploração global (diversidade populacional) e exploração local (vizinhanças);
- permitem incorporar conhecimento do domínio através de operadores específicos e inicialização heurística.

No contexto da otimização multiobjetivo, estas características tornam-nos especialmente eficazes na aproximação da Frente de Pareto, permitindo obter um conjunto diversificado de soluções não-dominadas numa única execução.

6.1 Representação das Soluções

Ao contrário de problemas clássicos abordados com algoritmos genéticos [6], em que as soluções são frequentemente representadas como vetores binários ou reais, neste trabalho cada indivíduo representa um **caminho num grafo dirigido multimodal**.

Assim, uma solução é codificada como:

$$x = (v_1, v_2, \dots, v_k)$$

onde:

- cada $v_i \in V$ representa um nó do grafo (paragem de metro, autocarro ou ponto de origem/destino);
- $v_1 = s$ (origem virtual) e $v_k = t$ (destino virtual);
- cada par consecutivo (v_i, v_{i+1}) corresponde a uma aresta válida $e \in E$ do grafo.

Esta representação baseada em caminhos garante que todas as soluções manipuladas são *válidas por construção*: não existem arestas inválidas ou ciclos, eliminando a necessidade de operadores de reparação ou penalizações estruturais.

Estrutura do Grafo Multimodal O grafo $G = (V, E)$ é construído integrando três redes de transporte da cidade do Porto:

- **Metro do Porto:** 13 estações conectadas através de dados GTFS¹, com tempos médios de viagem extraídos de horários reais;
- **STCP (autocarro):** 2500+ paragens com ligações baseadas em sequências de passagem de veículos;
- **Rede pedonal:** ligações a pé entre paragens com distância até 450 metros (raio de caminhabilidade), utilizando distância haversine corrigida por fator 1.2 para considerar a tortuosidade das ruas.

Cada aresta $e = (u, v) \in E$ armazena os seguintes atributos:

- t_e : tempo de viagem em minutos (média de horários GTFS para transporte público, calculado por velocidade 5 km/h para modo pedonal);
- d_e : distância percorrida em quilómetros;
- c_e : emissão de CO₂ em gramas, calculada como $c_e = d_e \times \text{fator}_{\text{modo}}$, onde:
 - $\text{fator}_{\text{metro}} = 40.0 \text{ g/km}$;
 - $\text{fator}_{\text{autocarro}} = 109.9 \text{ g/km}$;
 - $\text{fator}_{\text{pedonal}} = 0.0 \text{ g/km}$.
- m_e : modo de transporte $\in \{\text{metro}, \text{autocarro}, \text{pedonal}\}$.

Tratamento de Multimodalidade Uma particularidade das redes de transporte reais é a existência de múltiplas opções de deslocação entre os mesmos dois locais (por exemplo, percorrer um segmento de metro, autocarro ou a pé). Para modelar esta redundância, o grafo foi implementado utilizando a estrutura **MultiDiGraph** da biblioteca *NetworkX*, que suporta múltiplas arestas paralelas entre o mesmo par de nós (u, v) .

A desambiguação da aresta a utilizar em cada passo do algoritmo é realizada pela função auxiliar `get_edge()`. Esta função seleciona, de entre as arestas paralelas disponíveis, aquela que minimiza o custo escalar ponderado pelo vetor de pesos λ do subproblema atual:

$$e^* = \arg \min_{e \in E_{uv}} \left(\lambda_1 \cdot \frac{t_e}{t_{\max}} + \lambda_2 \cdot \frac{c_e}{c_{\max}} \right) \quad (9)$$

Este mecanismo dinâmico permite que o algoritmo alterne inteligentemente entre modos de transporte (ex.: preferir caminhar quando λ_{CO_2} é elevado) sem alterar a topologia base do grafo.

Avaliação de Soluções Dado um caminho $x = (v_1, \dots, v_k)$, a função `evaluate_path()` calcula:

$$T(x) = \sum_{i=1}^{k-1} t_{(v_i, v_{i+1})},$$

$$C(x) = \sum_{i=1}^{k-1} c_{(v_i, v_{i+1})},$$

¹General Transit Feed Specification

e ainda verifica:

- número de transferências entre modos: $n_{\text{trans}}(x)$;
- tempo total de caminhada: $T_{\text{walk}}(x)$.

Violações de restrições operacionais são penalizadas através de:

$$\text{violation}(x) = \begin{cases} 500 \times (n_{\text{trans}}(x) - 4), & \text{se } n_{\text{trans}}(x) > 4, \\ 0, & \text{caso contrário,} \end{cases} + \begin{cases} 100 \times (T_{\text{walk}}(x) - 90), & \text{se } T_{\text{walk}}(x) > 90 \text{ min,} \\ 0, & \text{caso contrário.} \end{cases}$$

Estas penalizações são adicionadas aos objetivos durante a seleção, desencorajando soluções impraticáveis sem removê-las completamente da população (permitindo que sirvam de "ponte" evolutiva para regiões do espaço de procura).

Implementação A construção do grafo encontra-se implementada em `graph.py`, através das funções:

- `load_stops()`: lê paragens de ficheiros GTFS;
- `load_gtfs_edges()`: constrói arestas de transporte público a partir de `stop_times.txt`, agregando múltiplas viagens por média aritmética;
- `add_walk_edges()`: gera ligações pedonais bidirecionais entre paragens próximas, incluindo penalização temporal de 2 minutos em transferências metro-autocarro;
- `build_graph()`: orquestra todo o processo e serializa o grafo resultante em `graph_base.gpickle`.

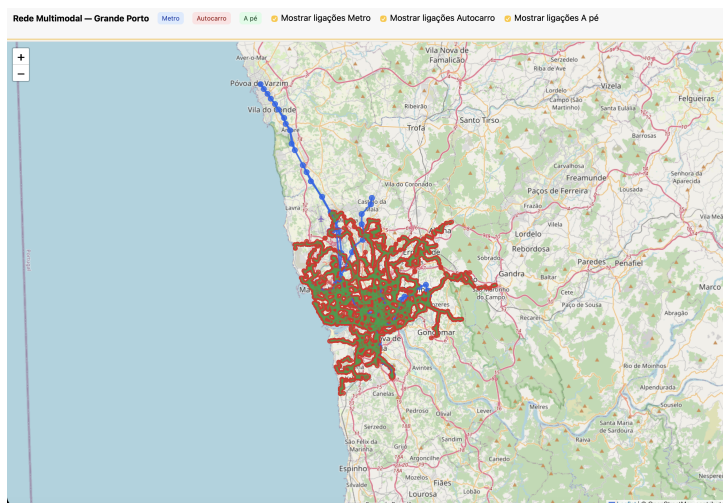


Figura 1: Excerto do grafo multimodal gerado no contexto do problema(disponível através do `export_graph.py`)

Nesta imagem podemos observar as diferentes opções possíveis, a verde as diferentes possibilidades de deslocação pedonal e a vermelho as deslocações de autocarro.

7 MOEA/D: Algoritmo Evolutivo Baseado em Decomposição

O algoritmo selecionado para a resolução do problema foi o **MOEA/D** (*Multi-Objective Evolutionary Algorithm based on Decomposition*) [10].

Ao contrário de abordagens baseadas em ordenação por dominância (como o NSGA-II), o MOEA/D decompõe o problema multiobjetivo em vários subproblemas escalares, resolvidos cooperativamente. Esta abordagem permite transformar a tarefa de aproximar a Frente de Pareto na resolução simultânea de N problemas de otimização escalar.

7.1 Decomposição do Problema e Vizinhança

A base do algoritmo reside na decomposição do problema multiobjetivo utilizando um conjunto de vetores de peso:

$$\Lambda = \{\lambda^1, \lambda^2, \dots, \lambda^N\}$$

onde cada $\lambda^i = (\lambda_1^i, \lambda_2^i)$ representa uma preferência específica entre os objetivos (tempo e CO_2).

Cada vetor λ^i define um subproblema escalar distinto. O MOEA/D resolve estes subproblemas simultaneamente, explorando a ideia de que subproblemas com pesos vizinhos têm, muito provavelmente, soluções ótimas semelhantes.

Vizinhança Para cada subproblema i , define-se um conjunto de vizinhos $B(i)$ contendo os T vetores de peso mais próximos de λ^i (em termos de distância Euclidiana). A otimização do subproblema i ocorre utilizando apenas informação genética proveniente deste conjunto de vizinhos, o que reduz a pressão seletiva global e mantém a diversidade da população.

7.2 Função de Agregação de Tchebycheff

Para converter o vetor de objetivos numa métrica escalar para cada subproblema, utilizou-se a função de Tchebycheff. A aptidão de uma solução x em relação ao vetor de pesos λ é dada por:

$$g^{te}(x|\lambda, z^*) = \max_{j \in \{1,2\}} \{\lambda_j \cdot |f_j(x) - z_j^*|\}$$

Esta função apresenta várias vantagens cruciais para este projeto:

- permite encontrar soluções em Frentes de Pareto não convexas (comum em problemas combinatórios);
- garante uma boa distribuição de soluções ao longo da frente;
- é computacionalmente eficiente.

Ponto de Referência Ideal O vetor $z^* = (z_1^*, z_2^*)$ representa o ponto de referência ideal, sendo atualizado dinamicamente durante a execução. Na implementação em `moea.py`, z^* armazena o melhor valor encontrado até ao momento para cada objetivo individualmente (mínimo tempo e mínimo CO_2 observados em toda a população).

7.3 Inicialização Heurística com Algoritmos de Caminho Mínimo

Uma vez definida a estrutura de decomposição, a qualidade da população inicial é determinante para o sucesso do algoritmo. Em vez de uma inicialização puramente aleatória, utiliza-se uma estratégia heurística baseada em algoritmos clássicos de procura.

Os objetivos desta inicialização são:

- gerar uma população inicial de elevada qualidade e diversificada;
- fornecer limites inferiores aproximados para os objetivos (tempo e CO_2);
- incorporar conhecimento do domínio através de soluções viáveis;
- acelerar a convergência do algoritmo evolucionário ao iniciar em regiões promissoras do espaço de procura.

Esta abordagem inspira-se em estratégias híbridas onde algoritmos exatos (Dijkstra) produzem "sementes" informadas para a meta-heurística.

Implementação da Inicialização A função `heuristic_initialization()` em `moead.py` concretiza este processo através dos seguintes passos detalhados:

1. **Geração de vetores de peso:** São gerados $N = 100$ vetores de pesos $\lambda^{(i)} = (\lambda_1^{(i)}, \lambda_2^{(i)})$ uniformemente distribuídos no espaço $[0, 1]^2$ com $\lambda_1 + \lambda_2 = 1$, mediante:

$$\lambda^{(i)} = \left(\frac{i}{N-1}, 1 - \frac{i}{N-1} \right), \quad i = 0, 1, \dots, N-1$$

Esta decomposição linear garante uma cobertura uniforme do *trade-off* entre os dois objetivos, desde o extremo do tempo até ao extremo do CO_2 .

2. **Normalização dos atributos:** Para evitar que objetivos com escalas maiores (como o CO_2 em gramas) dominem a busca, os atributos são normalizados pelos seus valores máximos estimados no grafo:

$$t_{\max} = \max_{e \in E} t_e, \quad c_{\max} = \max_{e \in E} c_e$$

3. **Procura de caminho mínimo ponderado:** Para cada vetor de pesos $\lambda^{(i)}$, executa-se o algoritmo de Dijkstra (via `nx.shortest_path()`) com uma função de custo linearizada:

$$\text{cost}(u, v) = \lambda_1^{(i)} \cdot \frac{t_{(u,v)}}{t_{\max}} + \lambda_2^{(i)} \cdot \frac{c_{(u,v)}}{c_{\max}}$$

Desta forma, $\lambda^{(0)} = (0, 1)$ minimiza apenas o CO_2 , enquanto $\lambda^{(N-1)} = (1, 0)$ minimiza apenas o tempo.

4. **Seleção de arestas em MultiDiGraph:** Após identificar a sequência de nós com `shortest_path()`, a função `evaluate_path()` seleciona, para cada par de nós, a aresta específica (ex: qual linha de autocarro) que minimiza o mesmo custo ponderado usado na busca. Isto garante consistência entre a estrutura do caminho encontrado e a sua avaliação real.

5. **Seed especial (Solução 100% pedonal):** A solução associada ao vetor $\lambda^{(0)}$ (extremo ecológico) é explicitamente substituída por uma solução exclusivamente pedonal, se viável. Esta rota é calculada num subgrafo contendo apenas arestas `modo="walk"`, garantindo que a Frente de Pareto inclua a opção de "zero emissões" desde a primeira geração.
6. **Completar população por mutação:** Caso o Dijkstra falhe para alguns vetores (ex: `NetworkXNoPath` em zonas desconexas para certos modos), a população é completada até N através da seleção aleatória de soluções existentes e aplicação do operador de mutação. Isto preserva a diversidade genética sem introduzir soluções aleatórias inválidas.

Impacto na Convergência Esta estratégia produz uma população onde todas as soluções são topologicamente válidas e já se encontram próximas da Frente de Pareto ótima. Os valores iniciais obtidos servem imediatamente para calibrar o ponto de referência z^* da função de Tchebycheff, permitindo que o MOEA/D se foque no refinamento e preenchimento de lacunas na frente de compromisso desde a primeira geração.

8 Operadores Genéticos Específicos para Grafos

Devido à natureza sequencial das soluções (caminhos em grafos), foi necessário definir operadores genéticos adaptados a sequências de nós, em vez de operadores clássicos aplicáveis a vetores binários ou reais.

8.1 Operador de Crossover por Nó Comum

O operador de crossover implementado em `moead.py` combina dois percursos pais através da seguinte estratégia:

1. Seja $p_1 = (v_1^{(1)}, \dots, v_{k_1}^{(1)})$ e $p_2 = (v_1^{(2)}, \dots, v_{k_2}^{(2)})$ dois caminhos pais.
2. Identificam-se nós comuns: $\mathcal{C} = p_1 \cap p_2$.
3. Se $|\mathcal{C}| < 3$ ou com probabilidade $1 - r_c$ (onde $r_c = 0.8$ é a taxa de crossover), um pai é retornado inalterado.
4. Caso contrário, seleciona-se aleatoriamente um nó $n^* \in \mathcal{C}$.
5. O filho é construído como:

$$\text{offspring} = p_1[1 : \text{índice}(n^*)] \parallel p_2[\text{índice}(n^*) : \text{fim}]$$

onde $\parallel$ denota concatenação.

Propriedades Este operador garante:

- **Validade topológica:** o filho é sempre um caminho válido no grafo (concatenação de caminhos válidos);
- **Preservação de subestruturas:** segmentos eficientes dos pais são mantidos;
- **Exploração multimodal:** combina diferentes estratégias de modos de transporte dos dois pais;
- **Diversidade controlada:** a exigência de $|\mathcal{C}| \geq 3$ evita cruzamentos entre soluções muito distintas.

8.2 Operador de Mutação com Recombinação de Segmentos

A mutação em `moead.py` é responsável pela exploração local e introdução de diversidade:

1. Com probabilidade $r_m = 0.8$ (`MUTATION_RATE`), processa-se a mutação; caso contrário, o caminho é retornado inalterado.
2. Selecionam-se dois índices aleatórios i, j com $0 \leq i < j \leq |p| - 1$.
3. Remove-se o subcaminho intermédio: $p[i : j]$ é descartado.
4. Reconstrói-se o subcaminho entre $p[i]$ e $p[j]$ mediante:

- **Modo exploratório** (durante evolução): executa-se Dijkstra escolhendo aleatoriamente entre minimizar tempo t_e ou CO₂ c_e :

$$\text{subcaminho} = \text{shortest_path}(G, p[i], p[j], \text{weight} \in \{t_e, c_e\})$$

- **Modo orientado** (durante inicialização): executa-se Dijkstra com função de custo ponderada e jitter:

$$\text{cost}(u, v) = \text{edge_score}(e_{uv}) \cdot (1 + \delta), \quad \delta \in [-0.05, 0.05]$$

onde o jitter δ introduz aleatoriedade controlada para escapar de mínimos locais.

5. O novo caminho é $p' = p[0 : i] \parallel \text{subcaminho} \parallel p[j + 1 :]$.
6. Remove-se nós duplicados consecutivos.

Justificação Esta estratégia combina:

- **Garantia de validade:** a reconstrução via Dijkstra assegura que o subcaminho gerado é sempre um caminho válido no grafo.
- **Compensação da Inicialização Heurística:** A taxa de mutação elevada ($r_m = 0.8$) foi deliberadamente escolhida para contrabalançar a convergência prematura induzida pela inicialização. Visto que a população inicial é gerada por algoritmos gulosos (Dijkstra), as soluções tendem a concentrar-se precocemente em torno de ótimos locais. Uma taxa de mutação agressiva força a perturbação destas estruturas, garantindo que o algoritmo continua a explorar rotas alternativas que a heurística determinística ignorou.
- **Exploração local guiada (fase inicial):** quando a população ainda não tem 100 indivíduos (não conseguimos criar o indivíduo `walk_only`), a mutação usa um custo ponderado pelos pesos λ_i do subproblema (modo orientado) e um jitter de $\pm 5\%$, o que ajuda a criar variações próximas das soluções heurísticas, mantendo o trade-off desejado.
- **Exploração ampla (fase evolutiva):** durante as gerações, a mutação usa um objetivo aleatório (minimizar tempo ou CO₂) em cada chamada (modo exploratório), introduzindo diversidade e evitando que a população fique presa em um único critério.

9 Atualização da População e Manutenção da Frente de Pareto

9.1 Mecanismo de Seleção Baseado em Vizinhanças (MOEA/D)

Ao contrário de algoritmos genéticos clássicos que utilizam seleção global por dominância, o MOEA/D implementado (no `moead.py`) utiliza um modelo de vizinhança:

1. Geração de vizinhanças no espaço de pesos:

$$\mathcal{N}(i) = \{j_1, j_2, \dots, j_K\} \quad \text{onde} \quad K = 20 \text{ e } \|w_i - w_j\| \text{ é mínimo}$$

Assim, subproblemas com trade-offs semelhantes formam vizinhanças, promovendo cooperação entre objetivos afins.

2. Para cada indivíduo i na população:

- Selecionam-se dois pais: $p_1 = \text{population}[i]$ e $p_2 = \text{population}[j]$ onde $j \in \mathcal{N}(i)$ (escolhido aleatoriamente).
- Aplicam-se operadores genéticos (crossover e mutação) para produzir um filho.
- O filho é avaliado com a função de agregação Tchebycheff orientada aos pesos de i .

3. Atualização de vizinhos: Para cada vizinho $j \in \mathcal{N}(i)$, calcula-se:

$$\text{scal}(\text{child}, j) = \max\{\lambda_1^{(j)}|T(\text{child}) - z_1|, \lambda_2^{(j)}|C(\text{child}) - z_2|\} + \text{violation}(\text{child})$$

$$\text{scal}(\text{population}[j], j) = \max\{\lambda_1^{(j)}|T(\text{population}[j]) - z_1|, \lambda_2^{(j)}|C(\text{population}[j]) - z_2|\} + \text{violation}(\text{population}[j])$$

Se $\text{scal}(\text{child}, j) < \text{scal}(\text{population}[j], j)$, o filho substitui o indivíduo j .

Este mecanismo reduz a pressão seletiva global (comparativamente a seleção por torneio global), permitindo que soluções subótimas globais mas especializadas em certos trade-offs sobrevivam e contribuam para a frente de Pareto.

9.2 Manutenção da Frente de Pareto com Penalização

A frente de Pareto é atualizada após cada geração (função `update_pareto()`) através de:

1. Para cada nova solução `sol`, verifica-se dominância penalizada:

$$f'(x) = (T(x) + \text{violation}(x), \quad C(x) + \text{violation}(x))$$

onde $\text{violation}(x)$ acarreta penalidades por excesso de transferências ou tempo de caminhada.

2. Uma solução x domina y (escreve-se $x \prec y$) se:

$$f'(x) \leq f'(y) \quad \text{e} \quad \exists i : f'_i(x) < f'_i(y)$$

3. Remove-se soluções dominadas; adiciona-se a nova solução e aplica-se poda por epsilon-dominance.

9.3 Poda por Epsilon-Dominance

Para evitar crescimento ilimitado da frente de Pareto e manter diversidade, a função `prune_pareto_epsilon()` remove soluções redundantes:

1. Ordena-se a frente por **compromisso normalizado**:

$$\text{score}(x) = \frac{T(x) - T_{\min}}{T_{\max} - T_{\min}} + \frac{C(x) - C_{\min}}{C_{\max} - C_{\min}}$$

Esta ordenação equilibrada evita viés para soluções rápidas ou ecológicas.

2. Retêm-se apenas soluções que diferem significativamente (epsilon-dominance):

$$\text{keep } x \text{ se } \nexists y \in \text{pruned} : |T(x) - T(y)| \leq \epsilon_t \text{ AND } |C(x) - C(y)| \leq \epsilon_c$$

onde $\epsilon_t = 0.3$ min e $\epsilon_c = 1.0$ g são tolerâncias de redundância.

3. Este critério garante que a frente mantém soluções estruturalmente distintas, não apenas Pareto-ótimas matematicamente.

10 Metodologia Experimental

A avaliação foca-se na aproximação da Frente de Pareto, analisando convergência, diversidade e qualidade dos compromissos.

10.1 Cenários de Teste

Foram gerados automaticamente 12 cenários (3 por nível de dificuldade) a partir do grafo multimodal do Grande Porto, controlando a distância aérea origem–destino por bandas:

- **Fácil:** 0.5–2.5 km (3 cenários)
- **Médio:** 2.5–5.0 km (3 cenários)
- **Difícil:** 5.0–10.0 km (3 cenários)
- **Muito Difícil:** 10.0–20.0 km (3 cenários)

Os cenários são guardados em `output/scenarios.csv` e `.pkl`, permitindo replicação e inspeção.

10.2 Configuração Experimental

Todos os testes usam os mesmos hiperparâmetros para comparabilidade:

Parâmetro	Valor
Tamanho da população (N)	100
Número de gerações	50
Número de vizinhos (T)	20
Probabilidade de crossover	0.8
Probabilidade de mutação	0.8
Função de agregação	Tchebycheff
Penalização de transferências	$500 \times$ excedente acima de 4
Penalização de tempo a pé	$100 \times$ excedente acima de 90 min

Tabela 1: Parâmetros utilizados nos testes experimentais.

As penalizações mantêm soluções viáveis mas desincentivam trajetos com demasiadas trocas ou caminhada excessiva.

Parâmetro do Grafo	Valor
Raio de conexão pedonal	0.45 km
Velocidade de caminhada	5.0 km/h
Tempo fixo de transbordo	2.0 min
Fator de emissão CO_2 (Metro)	40.0 g/km
Fator de emissão CO_2 (Autocarro)	109.9 g/km

Tabela 2: Parâmetros de construção do grafo multimodal e fatores de emissão.

11 Métricas de Avaliação

11.1 Convergência

Monitorizamos por geração:

- Mínimos de tempo e CO₂ na frente de Pareto;
- Hipervolume 2D (ref. $z = 1,1 \times (\max T, \max C)$) — quanto maior, melhor;
- Tamanho da frente (número de soluções não-dominadas).

11.2 Diversidade

A diversidade é promovida e analisada através de:

- Poda por *epsilon-dominance* ($\epsilon_t = 0,3$ min, $\epsilon_c = 1,0$ g) para evitar soluções redundantes;
- Vizinhanças em espaço de pesos (MOEA/D) que espalham soluções ao longo dos trade-offs;
- Dispersão visual das soluções (tempo vs. CO₂) e cobertura entre extremos.

12 Resultados Experimentais

Os resultados apresentados nesta secção correspondem a três cenários de mobilidade urbana no Porto, um por nível de dificuldade (distância aérea crescente):

- Cenário Fácil (exemplo 0.5–3.5 km);
- Cenário Médio (exemplo 4–7 km);
- Cenário Difícil (exemplo 8.5–17 km).

12.1 Frente de Pareto Obtida

A Figura 2 mostra as Frentes de Pareto aproximadas obtidas após 50 gerações do MOEA/D, com a identificação explícita de três soluções representativas em cada cenário.

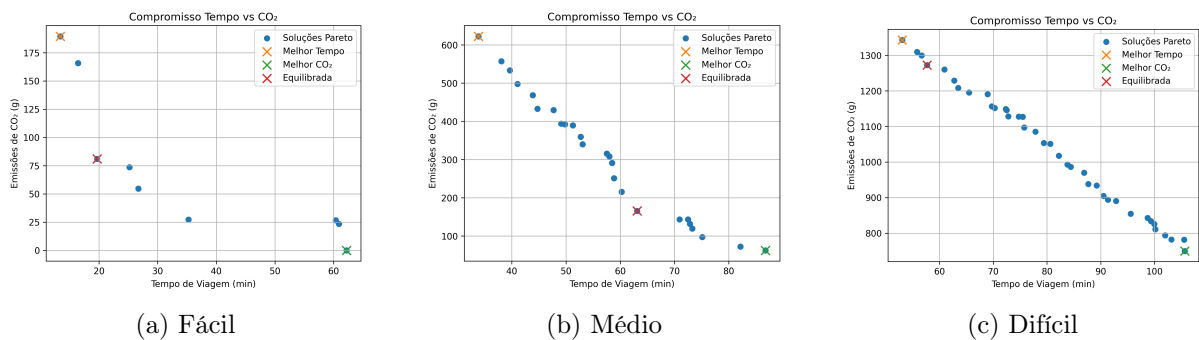


Figura 2: Compromisso entre tempo de viagem e emissões de CO₂ para um caso de cada nível de dificuldade.

Mantém-se a natureza conflituosa do problema: as soluções mais rápidas usam mais transporte motorizado (emissões elevadas), enquanto a solução ambientalmente ótima é mais longa e dominada por deslocação pedonal. A solução equilibrada situa-se na região intermédia da frente, representando um compromisso temporal-ambiental.

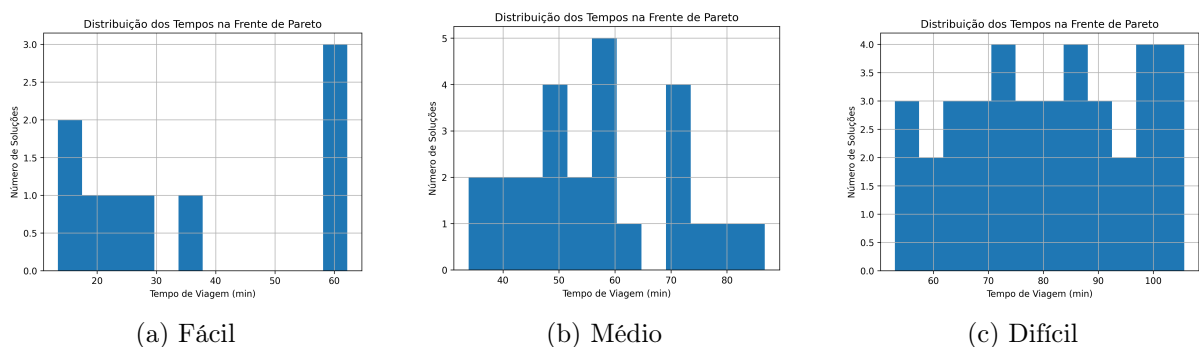


Figura 3: Distribuição dos tempos das soluções não dominadas para um caso de cada nível de dificuldade.

A Figura 3 evidencia dois grupos típicos: soluções rápidas (maior uso de transporte público motorizado) e soluções mais lentas com elevada componente pedonal. A diversidade estrutural mantém-se em todos os níveis, apesar de os cenários mais longos mostrarem dispersão temporal maior.

12.2 Soluções Representativas

Para facilitar a leitura, a Tabela 3 mostra três perfis representativos (Rápido, Equilibrado, Ecológico) para o cenário de dificuldade média (exemplo usado nas Figuras 2 e 3, segundo painel).

Perfil	Tempo (min)	CO_2 (g)	Modo Dominante
Rápido	33.9	622	Bus > Metro > Walk
Equilibrado	63.1	166	Walk > Metro > Bus
Ecológico	86.8	62	Walk > Metro

Tabela 3: Soluções representativas para o cenário médio. Substitui pelos valores do caso selecionado.

13 Análise da Convergência

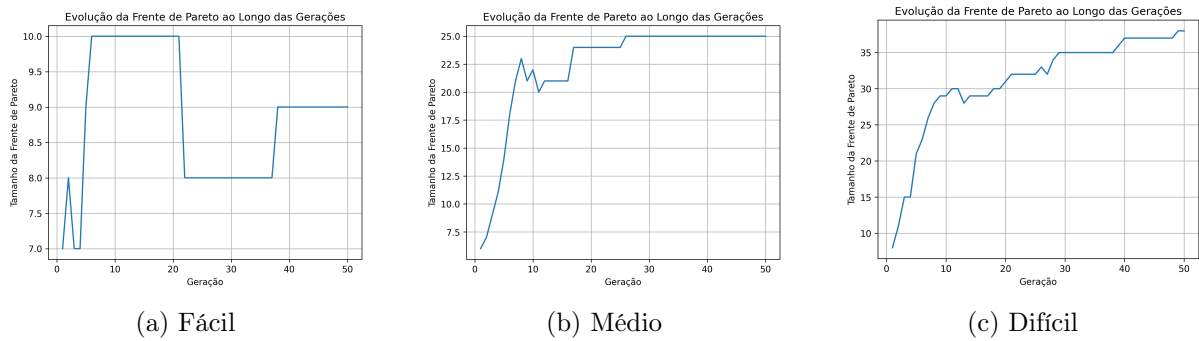


Figura 4: Evolução do tamanho da Frente de Pareto ao longo das gerações para um caso de cada nível de dificuldade.

O crescimento rápido nas primeiras gerações e a estabilização por volta das 30–35 gerações repetem-se nos três níveis, confirmando a convergência estável do MOEA/D.

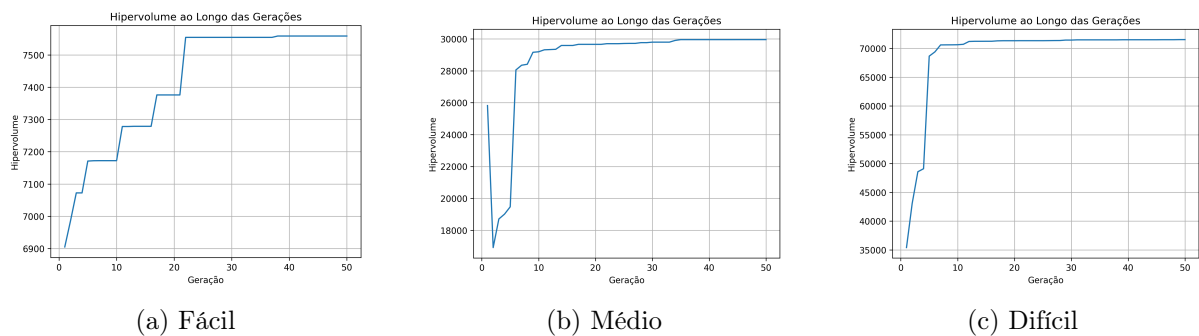


Figura 5: Evolução do hipervolume ao longo das gerações para um caso de cada nível de dificuldade.

O hipervolume sobe de forma acentuada nas gerações iniciais e estabiliza após 30 iterações, indicando aproximação consistente à Frente de Pareto em todos os níveis. A mutação baseada em Dijkstra continua a produzir melhorias pontuais mesmo em fases tardias.

14 Discussão dos Resultados

Os resultados obtidos demonstram que o MOEA/D é uma abordagem adequada para o planejamento de percursos multimodais em redes urbanas reais.

14.1 Vantagens da Abordagem

Entre as principais vantagens, destacam-se:

- capacidade de gerar múltiplos compromissos numa única execução;
- integração eficaz de conhecimento do domínio;
- boa cobertura da Frente de Pareto;
- robustez face à complexidade do grafo.

14.2 Limitações

Apesar dos resultados positivos, existem algumas limitações:

- custo computacional associado à avaliação de caminhos longos;
- dependência da qualidade da inicialização heurística;
- ausência de dados em tempo real (ex.: atrasos, congestionamento).

14.3 Ameaças à Validade

As principais ameaças à validade dos resultados incluem:

- utilização de fatores médios de emissão de CO_2 ;
- avaliação baseada num número limitado de cenários;
- ausência de comparação direta com outros algoritmos multiobjetivo (ex.: NSGA-II).

15 Conclusões e Trabalho Futuro

Neste trabalho foi desenvolvido um sistema de planeamento de percursos multimodais para o Grande Porto, formulado como um problema de otimização multiobjetivo e resolvido através do algoritmo MOEA/D.

Os resultados demonstram que a abordagem proposta é capaz de gerar Frentes de Pareto bem distribuídas, oferecendo ao utilizador final diferentes opções de compromisso entre tempo de viagem e impacto ambiental.

Como trabalho futuro, propõe-se:

- inclusão de restrições adicionais (ex.: limite máximo de caminhada);
- integração de dados em tempo real;
- comparação experimental com outros algoritmos multiobjetivo;
- extensão do modelo para outros critérios, como custo monetário ou conforto.

A Visualização da Rede Multimodal

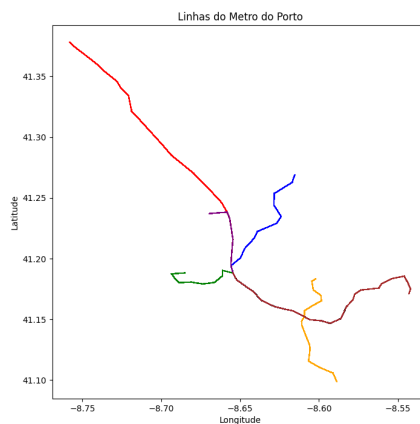


Figura 6: Representação geográfica das linhas do Metro do Porto utilizadas na construção do grafo multimodal.

A Figura 6 apresenta a geometria das linhas do Metro do Porto, permitindo visualizar a cobertura espacial deste modo de transporte no contexto da Área Metropolitana. Esta visualização confirma a coerência geográfica dos dados utilizados e evidencia o papel estruturante do Metro nos percursos de menor tempo identificados pelo algoritmo.

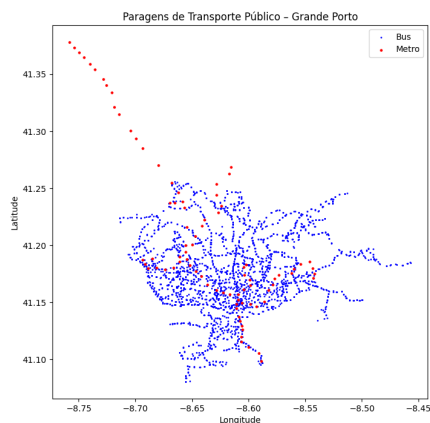


Figura 7: Distribuição espacial das paragens de transporte público (Metro e Autocarro) no Grande Porto.

A Figura 7 mostra a distribuição espacial das paragens de Metro e Autocarro consideradas no modelo. Observa-se uma elevada densidade de paragens de autocarro na zona central, contrastando com a estrutura mais linear da rede de Metro.

Esta heterogeneidade justifica a necessidade de um modelo multimodal e reforça a importância das ligações pedonais para assegurar conectividade entre modos.

Referências

- [1] Câmara Municipal do Porto. *Infraestruturas e Mobilidade — Dados GTFS*. https://opendata.porto.digital/dataset/?q=Infraestruturas+e+Mobilidade&res_format=GTFS. Acedido em 2025. 2024.
- [2] Carlos A. Coello Coello. «Evolutionary Multi-Objective Optimization: A Historical View of the Field». Em: *IEEE Computational Intelligence Magazine* 1.1 (2006), pp. 28–36. DOI: [10.1109/MCI.2006.1597059](https://doi.org/10.1109/MCI.2006.1597059).
- [3] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons, 2001. ISBN: 9780471873396.
- [4] Kalyanmoy Deb et al. «A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II». Em: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), pp. 182–197. DOI: [10.1109/4235.996017](https://doi.org/10.1109/4235.996017).
- [5] Agoston E. Eiben e J. E. Smith. *Introduction to Evolutionary Computing*. Springer, 2003. ISBN: 9783540401841.
- [6] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989. ISBN: 9780201157673.
- [7] Joshua Knowles e David Corne. «Approaches to Multiobjective Optimization». Em: *Evolutionary Computation* 8.2 (2006), pp. 125–169.
- [8] Metro do Porto. *Política de Sustentabilidade e Ambiente*. <https://www.metrodoporto.pt/pages/358>. Relatório institucional. 2024.
- [9] STCP. *Política Energética e Sustentabilidade*. <https://www.stcp.pt/pt/institucional/sustentabilidade/politica-energetica/>. Relatório institucional. 2024.
- [10] Qingfu Zhang e Hui Li. «MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition». Em: *IEEE Transactions on Evolutionary Computation* 11.6 (2007), pp. 712–731. DOI: [10.1109/TEVC.2007.892759](https://doi.org/10.1109/TEVC.2007.892759).